Design Module:

Prompt Strategy: Zero Shot

Prompt:

Write a complete Verilog implementation of a 16-bit signed adder using
STRICTLY BEHAVIORAL MODELING ONLY.

STRICT BEHAVIORAL RULE:
The design module must contain ONLY always @(*) procedural blocks with
blocking assignments (=). No assign statements, no structural instantiation,
no gate primitives, no module instantiation of any kind is allowed inside
the design module. All outputs driven from always blocks MUST be declared as reg.

DESIGN MODULE REQUIREMENTS:
- Module name: adder16_beh
- input signed [15:0] A
- input signed [15:0] B
- output reg signed [16:0] sum
- output reg ovf
- Inside always @(*):
    sum = $signed({A[15],A}) + $signed({B[15],B});
    ovf = sum[16] ^ sum[15];

TESTBENCH REQUIREMENTS:
Write tb_adder16 that instantiates adder16_beh and tests it using
ALL FOUR loop types:

FOR LOOP — iterate over these 20 corner case pairs (A,B):
  (0,0),(1,0),(0,1),(-1,0),(0,-1),
  (1,1),(-1,-1),(1,-1),
  (32767,0),(-32768,0),
  (32767,1),(-32768,-1),
  (32767,32767),(-32768,-32768),
  (32767,-32768),(-32768,32767),
  (16384,16384),(-16384,-16384),
  (32766,1),(-32767,-1)
  Per iteration: apply A,B → #10 → compute expected=$signed(A)+$signed(B)
  → compare $signed(sum) → print PASS or FAIL with all values

WHILE LOOP:
  Run while test_count < 100
  A = $signed($random); B = $signed($random); #10;
  Compare result, track error_count, print PASS or FAIL each iteration

REPEAT LOOP:
  Repeat 50 times
  A = $signed($random); B = $signed($random); #10;
  $display A, B, sum, ovf each iteration

FOREVER LOOP:
  reg clk = 0;
  Toggle clk every 5ns
  Terminate with $finish at 1000ns in a separate initial block

Print final error summary before $finish.
Use %0d for signed decimal and %b for binary flags in all $display calls.

...

Design Module:

```
`timescale 1ns/1ps

//=====================================================
// 16-bit Signed Adder (Behavioral Modeling Only)
//=====================================================
module adder16_beh(
input signed [15:0] A,
input signed [15:0] B,
output reg signed [16:0] sum,
output reg ovf
);

always @(*) begin
sum = $signed({A[15],A}) + $signed({B[15],B});
ovf = sum[16] ^ sum[15];
end

endmodule
```

...

Prompt Strategy: Few Shot

Prompt:

I will show you examples of behavioral Verilog. You will extend them into a complete 16-bit signed adder with testbench.

STRICT BEHAVIORAL RULE:
ALL design modules must use ONLY always @(*) with blocking assignments (=).
NO assign statements. NO structural instantiation. NO gate primitives.
NO always @(posedge clk) in design modules.
All outputs driven from always blocks must be declared as output reg.

EXAMPLE 1 — 4-bit behavioral adder:

```
module adder4_beh(
  input  signed [3:0]  A, B,
  output reg signed [4:0] sum
);
  always @(*) begin
    sum = $signed({A[3],A}) + $signed({B[3],B});
  end
endmodule
```

EXAMPLE 2 — 8-bit behavioral adder with overflow flag:

```
module adder8_beh(
  input  signed [7:0]  A, B,
  output reg signed [8:0] sum,
  output reg ovf
);
  always @(*) begin
    sum = $signed({A[7],A}) + $signed({B[7],B});
    ovf = sum[8] ^ sum[7];
  end
endmodule
```

EXAMPLE 3 — Behavioral adder with conditional logic inside always:

```
module adder8_cond(
  input  signed [7:0]  A, B,
  output reg signed [8:0] sum,
  output reg ovf,
  output reg zero_flag
);
  always @(*) begin
    sum       = $signed({A[7],A}) + $signed({B[7],B});
    ovf       = sum[8] ^ sum[7];
    zero_flag = (sum == 9'b0) ? 1'b1 : 1'b0;
  end
endmodule
```

EXAMPLE 4 — Testbench for loop pattern:

```
initial begin
  for (i = 0; i < 8; i = i + 1) begin
    A = i; B = ~i + 1; #10;
    $display("A=%0d B=%0d Sum=%0d", $signed(A), $signed(B), $signed(sum));
  end
end
```

NOW DO THE FOLLOWING:
1. Scale EXAMPLE 2 to a 16-bit signed behavioral adder (adder16_beh):
   - always @(*) only — no assign, no structural code
   - sum = $signed({A[15],A}) + $signed({B[15],B});
   - ovf = sum[16] ^ sum[15];
   - Add zero_flag = (sum == 17'b0) from EXAMPLE 3
   - All outputs declared as output reg

2. Build testbench from EXAMPLE 4 with ALL four loop types:
   - for loop: 20 corner case pairs with PASS/FAIL check
   - while loop: 100 iterations with $signed($random), error tracking
   - repeat loop: 50 iterations with $signed($random), display results
   - forever loop: clk toggle every 5ns, $finish at 1000ns
   PASS/FAIL check in every loop. Display A, B, sum, ovf, zero_flag each time.

...

Design Module:

```
//=======================================================
// 16-bit Signed Behavioral Adder
//=======================================================
module adder16_beh(
input signed [15:0] A,
input signed [15:0] B,
output reg signed [16:0] sum,
output reg ovf,
output reg zero_flag
);

always @(*) begin
sum = $signed({A[15],A}) + $signed({B[15],B});
ovf = sum[16] ^ sum[15];
zero_flag = (sum == 17'b0) ? 1'b1 : 1'b0;
end

endmodule
```

...

Prompt Strategy: Chain of Thought

Prompt:

Think step by step through every decision before writing any Verilog code.

STRICT BEHAVIORAL RULE:
The design module must use ONLY always @(*) with blocking assignments.
No assign, no structural instantiation, no gate primitives anywhere in the design.
All outputs driven from always blocks must be declared as output reg.

STEP 1 — UNDERSTAND BEHAVIORAL MODELING:
Think: What does behavioral modeling mean in Verilog?
It describes WHAT a circuit does using procedural always blocks, not HOW it is physically built. always @(*) automatically includes all inputs in the sensitivity list and re-evaluates whenever any input changes — correct for combinational logic.
Why not assign? assign only allows a single continuous expression. always @(*) allows if/else, case, multiple statements, local variable computation — far more expressive for describing behavior.

STEP 2 — UNDERSTAND REG VS WIRE:
Think: Why must sum and ovf be declared as reg?
In Verilog, any signal assigned inside a procedural block (always, initial) must be declared as reg. Wire is only for signals driven by assign or module output ports.
If sum is declared as wire and driven from always, the simulator will throw an error.

STEP 3 — UNDERSTAND BLOCKING VS NON-BLOCKING:
Think: What is the difference between = and <=?
Blocking (=): executes immediately in sequence — correct for combinational always @(*)
Non-blocking (<=): schedules update at end of time step — correct for clocked always @(posedge clk)
For a combinational adder, ALWAYS use blocking (=) inside always @(*).

STEP 4 — UNDERSTAND SIGN EXTENSION:
Think: Why use $signed({A[15],A}) instead of just A?
A is 16 bits. The sum output is 17 bits. Simply writing A + B would do unsigned 16-bit addition. We need to sign-extend A and B to 17 bits first.
{A[15],A} concatenates the sign bit MSB with A making a 17-bit value.
$signed() tells Verilog to treat this 17-bit value as signed.
Result: $signed({A[15],A}) + $signed({B[15],B}) gives correct 17-bit signed sum.

STEP 5 — UNDERSTAND OVERFLOW DETECTION:
Think: How does ovf = sum[16] ^ sum[15] detect signed overflow?
sum[16] is the extended sign bit (carry into sign position).
sum[15] is the actual result sign bit.

If they differ, overflow occurred — two positives gave negative or vice versa.
Example: 32767 + 1 = 32768 → in 17-bit binary sum[16]=0, sum[15]=1 → XOR=1 → overflow.

STEP 6 — WRITE THE BEHAVIORAL MODULE:
Write adder16_beh with:
- input signed [15:0] A, B
- output reg signed [16:0] sum
- output reg ovf
- always @(*) begin
  sum = $signed({A[15],A}) + $signed({B[15],B});
  ovf = sum[16] ^ sum[15];
  end

STEP 7 — WRITE THE TESTBENCH:
Think through all corner cases: 0+0, max+0, max+1 (overflow), min+(-1) (overflow), max+max, min+min, max+min, -1+1, random positives, random negatives, mixed signs.

Write tb_adder16 with:
- for loop: 20 corner case pairs, #10 delay, PASS/FAIL per iteration
- while loop: 100 random iterations, error_count tracking
- repeat loop: 50 random iterations, display A, B, sum, ovf
- forever loop: clk every 5ns, $finish at 1000ns

STEP 8 — VERIFY:
Check: Only always @(*) in design? sum and ovf are reg? signed on all declarations? 17-bit output? Blocking assignments used? All 4 loops present? $finish present?
Fix all issues before outputting final code.

...

Design Module:

```
//=================================================
// 16-bit Signed Behavioral Adder
//=================================================
module adder16_beh(
input signed [15:0] A,
input signed [15:0] B,
output reg signed [16:0] sum,
output reg ovf
);

always @(*) begin
sum = $signed({A[15], A}) + $signed({B[15], B});
ovf = sum[16] ^ sum[15];
end
```

endmodule

...

Prompt Strategy: Role Prompting

Prompt:

You are a senior RTL verification engineer and Verilog instructor with 15 years of experience writing behavioral RTL for FPGA synthesis and simulation.

Your team follows a strict behavioral coding standard:
ALL combinational design logic must be written inside always @(*) blocks using blocking assignments (=) ONLY. No assign statements, no structural instantiation, no gate primitives are permitted inside any design module. Every output driven from an always block must be declared as output reg.

A junior engineer needs a golden behavioral reference implementation of a 16-bit signed adder. Write it as you would for a professional code review submission.

DESIGN MODULE — adder16_beh:
  Ports:
    input signed [15:0] A
    input signed [15:0] B
    output reg signed [16:0] sum
    output reg ovf
  Implementation:
    always @(*) begin
      // Sign-extend both inputs to 17 bits before adding
      // Prevents data loss and correctly handles two's complement
      sum = $signed({A[15],A}) + $signed({B[15],B});
      // Overflow: extended sign bit disagrees with result sign bit
      ovf = sum[16] ^ sum[15];
    end
  Rules:
    - NO assign statements inside this module
    - NO structural instantiation or gate primitives
    - NO always @(posedge clk)
    - NO non-blocking assignments (<=)
  Add a professional header comment block explaining:
    - Module purpose
    - Behavioral modeling rationale
    - Signed overflow detection method
    - Port descriptions

TESTBENCH — tb_adder16:
  Signal declarations:
    reg signed [15:0] A, B
    wire signed [16:0] sum
    wire ovf
    integer i, test_count, error_count
    reg signed [16:0] expected

  FOR loop:
    Test all 20 boundary pairs:
    (0,0),(1,0),(0,1),(-1,0),(0,-1),(1,1),(-1,-1),(1,-1),
    (32767,0),(-32768,0),(32767,1),(-32768,-1),
    (32767,32767),(-32768,-32768),(32767,-32768),(-32768,32767),
    (16384,16384),(-16384,-16384),(32766,1),(-32767,-1)
    Apply → #10 → compute expected → PASS/FAIL display

  WHILE loop:
    100 iterations, $signed($random) for A and B
    Track error_count, display PASS or FAIL with all values

  REPEAT loop:
    50 iterations, $signed($random) inputs
    Display A, B, sum, ovf every iteration

  FOREVER loop:
    reg clk = 0;
    Toggle every 5ns
    Separate initial block: #2000 $finish;

  END: $display("Total Errors: %0d", error_count);

Write clean, professional, fully commented Verilog. Every significant line must have an inline comment explaining its purpose.

...

Design Module:

```
module adder16_beh
(
input signed [15:0] A, // Signed operand A
input signed [15:0] B, // Signed operand B
output reg signed [16:0] sum, // 17-bit signed result
output reg ovf // Overflow flag
);

always @(*) begin // Combinational logic block
```

sum = $signed({A[15],A}) + $signed({B[15],B}); // Sign-extend and add

ovf = sum[16] ^ sum[15]; // Detect signed overflow

end

endmodule

...

Prompt Strategy: Instruction Format

Prompt:

TASK:
Write a 16-bit signed adder in Verilog using STRICTLY BEHAVIORAL MODELING ONLY.

ABSOLUTE BEHAVIORAL RULE:
The design module must contain ONLY always @(*) blocks with blocking assignments (=).
No assign statements. No structural instantiation. No gate primitives.
No always @(posedge clk) in the design module.
Every output driven by an always block MUST be declared as output reg.

DELIVER IN EXACTLY THIS FORMAT:

═══
SECTION 1: BEHAVIORAL DESIGN MODULE
═══
Module name : adder16_beh
Ports :
input signed [15:0] A
input signed [15:0] B
output reg signed [16:0] sum
output reg ovf
Logic : always @(*) ONLY — blocking assignments ONLY
Key statements:
sum = $signed({A[15],A}) + $signed({B[15],B});
ovf = sum[16] ^ sum[15];
Forbidden : assign, structural, gate primitives, <=, always @(posedge clk)
[Write complete Verilog module here]

═══
SECTION 2: TESTBENCH
═══
Module name : tb_adder16

DUT instance : adder16_beh dut(.A(A),.B(B),.sum(sum),.ovf(ovf));
Declarations :
  reg signed [15:0] A, B
  wire signed [16:0] sum
  wire ovf
  integer i, test_count, error_count
  reg signed [16:0] expected

--- FOR LOOP ---
Iterate over exactly 20 pairs:
(0,0),(1,0),(0,1),(-1,0),(0,-1),(1,1),(-1,-1),(1,-1),
(32767,0),(-32768,0),(32767,1),(-32768,-1),
(32767,32767),(-32768,-32768),(32767,-32768),(-32768,32767),
(16384,16384),(-16384,-16384),(32766,1),(-32767,-1)
Action per iteration:
  Apply A,B → wait #10
  expected = $signed(A) + $signed(B)
  if match: PASS display with test#, A, B, expected, got, ovf
  if mismatch: FAIL display + increment error_count

--- WHILE LOOP ---
  test_count = 0;
  while (test_count < 100) begin
    A=$signed($random); B=$signed($random); #10;
    expected=$signed(A)+$signed(B);
    // Compare, display PASS/FAIL, track error_count
    test_count=test_count+1;
  end

--- REPEAT LOOP ---
  repeat(50) begin
    A=$signed($random); B=$signed($random); #10;
    $display("REPEAT A=%0d B=%0d sum=%0d ovf=%b",
             $signed(A),$signed(B),$signed(sum),ovf);
  end

--- FOREVER LOOP ---
  reg clk=0;
  initial forever #5 clk=~clk;
  initial #2000 $finish;

--- END ---
  $display("Simulation done. Errors: %0d", error_count);
  $finish;

...

Design Module:

```
module adder16_beh(
    input  signed [15:0] A,
    input  signed [15:0] B,
    output reg signed [16:0] sum,
    output reg ovf
);

always @(*) begin
    sum = $signed({A[15], A}) + $signed({B[15], B});
    ovf = sum[16] ^ sum[15];
end

endmodule
```

...

Prompt Strategy: Negative Prompting

Prompt:

Write a 16-bit signed adder in Verilog using behavioral modeling only,
with a comprehensive testbench covering all loop types.

STRICTLY DO NOT VIOLATE ANY OF THE FOLLOWING RULES:

——————————————————————————————————————

DESIGN MODULE — BEHAVIORAL VIOLATIONS:

——————————————————————————————————————

DO NOT use assign statements inside the design module.
DO NOT use structural module instantiation inside the design module.
DO NOT use gate primitives (and, or, xor, nand, nor, not, buf).
DO NOT use always @(posedge clk) — this is combinational, use always @(*) only.
DO NOT use non-blocking assignments (<=) inside always @(*).
DO NOT declare sum or ovf as wire — they MUST be output reg.
DO NOT write sum = A + B without sign extension — use:
    sum = $signed({A[15],A}) + $signed({B[15],B});
DO NOT omit the overflow flag — output reg ovf must be computed as:
    ovf = sum[16] ^ sum[15];
DO NOT use a 16-bit output — sum must be output reg signed [16:0].
DO NOT omit the signed keyword from any port or signal declaration.

——————————————————————————————————————

TESTBENCH — COVERAGE VIOLATIONS:

━━

DO NOT use only one loop type — all four must be present: for, while, repeat, forever.
DO NOT skip overflow corner cases — explicitly test:
    32767+1, -32768+(-1), 32767+32767, -32768+(-32768).
DO NOT use $random without $signed() — use $signed($random) for signed values.
DO NOT skip #delay between stimulus and output check — minimum #10 required.
DO NOT skip PASS/FAIL checking — compare expected=$signed(A)+$signed(B) vs $signed(sum).
DO NOT leave the forever loop unterminated — $finish must exist in a separate initial block.
DO NOT skip the error summary — print total error_count at end of simulation.
DO NOT use wrong display format — use %0d for signed decimal, %b for binary flags.

Now write the complete correct Verilog following all rules strictly.

...

Design Module:

```
module signed_adder_16 (
input signed [15:0] A,
input signed [15:0] B,
output reg signed [16:0] sum,
output reg ovf
);

always @(*) begin
sum = $signed({A[15],A}) + $signed({B[15],B});
ovf = sum[16] ^ sum[15];
end

endmodule
```

...

Prompt Strategy: Constraints First

Prompt:

READ ALL CONSTRAINTS COMPLETELY BEFORE WRITING ANY CODE:

━━
DESIGN CONSTRAINTS
━━

[C1] BEHAVIORAL ONLY: adder16_beh must contain EXCLUSIVELY always @(*) blocks with blocking assignments. No exceptions.

[C2] FORBIDDEN IN DESIGN MODULE:
- assign statements
- structural module instantiation
- gate primitives (and, or, xor, etc.)
- always @(posedge clk)
- non-blocking assignments (<=)

[C3] ALWAYS BLOCK TYPE: Use always @(*) for combinational sensitivity.
This re-evaluates whenever A or B changes.

[C4] BLOCKING ASSIGNMENT: Use = inside always @(*).
Sequential execution order matters — compute sum before ovf.

[C5] OUTPUT REG: Declare both outputs as reg:
output reg signed [16:0] sum
output reg ovf
Reason: signals driven from always blocks must be reg in Verilog.

[C6] SIGN EXTENSION: Inside always block use exactly:
sum = $signed({A[15],A}) + $signed({B[15],B});
Reason: prevents unsigned truncation, correctly handles two's complement.

[C7] OVERFLOW FLAG: Compute after sum, inside same always block:
ovf = sum[16] ^ sum[15];
Reason: detects when extended sign bit disagrees with result sign bit.

[C8] PORT WIDTHS:
input signed [15:0] A — range -32768 to +32767
input signed [15:0] B — range -32768 to +32767
output reg signed [16:0] sum — 17 bits to capture signed overflow
output reg ovf — 1-bit signed overflow flag

[C9] SIGNED KEYWORD: Must appear on every port, reg, and wire declaration.

TESTBENCH CONSTRAINTS

[C10] FOR LOOP: Test exactly these 20 pairs in order:
(0,0),(1,0),(0,1),(-1,0),(0,-1),(1,1),(-1,-1),(1,-1),
(32767,0),(-32768,0),(32767,1),(-32768,-1),
(32767,32767),(-32768,-32768),(32767,-32768),(-32768,32767),

(16384,16384),(-16384,-16384),(32766,1),(-32767,-1)

[C11] WHILE LOOP: test_count < 100. Use $signed($random) for both A and B.
Track error_count. Display PASS or FAIL with full values each iteration.

[C12] REPEAT LOOP: Exactly 50 iterations. $signed($random) inputs.
Display A, B, sum, ovf every iteration.

[C13] FOREVER LOOP: Toggle clk every 5ns.
Terminate via $finish at 2000ns in a SEPARATE initial block.

[C14] PASS/FAIL: Compute expected = $signed(A) + $signed(B) in testbench.
Compare $signed(sum) vs expected. PASS if equal, FAIL + increment error_count if not.

[C15] DISPLAY FORMAT:
%0d for all signed decimal values
%b for ovf flag
Include: test number, A, B, expected, actual, ovf, status

[C16] FINAL SUMMARY:
$display("═══ Simulation Complete. Total Errors: %0d ═══", error_count);

NOW write complete Verilog satisfying ALL constraints C1 through C16.

...

Design Module:

```
module adder16_beh (
input signed [15:0] A,
input signed [15:0] B,
output reg signed [16:0] sum,
output reg signed ovf
);

always @(*) begin
sum = $signed({A[15],A}) + $signed({B[15],B});
ovf = sum[16] ^ sum[15];
end

endmodule
```

...

Prompt Strategy: Self-Planning Prompt

Prompt:

INSTRUCTION: Before writing any Verilog code, produce a complete written PLAN answering all questions below. Then execute that plan to produce final code.

STRICT BEHAVIORAL RULE:
adder16_beh must use ONLY always @(*) with blocking assignments (=).
No assign, no structural, no gate primitives. All driven outputs must be reg.

PLANNING PHASE — answer all 12 questions:

Q1. What is the exact module port signature including reg/wire/signed declarations?
Q2. Why must sum and ovf be declared as output reg and not output wire?
Q3. What is always @(*) and why is it the correct construct for a combinational adder?
Q4. What is the difference between = and <=? Which is correct here and why?
Q5. What does $signed({A[15],A}) do? Trace it for A=32767 and A=-1.
Q6. How does ovf = sum[16]^sum[15] detect overflow?
Trace manually for: 32767+1, -32768+(-1), 100+200.
Q7. What are the 20 corner case pairs for the for loop and why is each important?
Q8. What distinct verification purpose does each loop type serve?
(for = ?, while = ?, repeat = ?, forever = ?)
Q9. How is expected computed in the testbench? What type should it be declared as?
Q10. How should A, B, sum, ovf be declared in the testbench (reg vs wire)?
Q11. How do I prevent the forever loop from running indefinitely?
Q12. What final summary output should the testbench print?

Write complete answers to Q1–Q12 as your PLAN section.

EXECUTION PHASE — write based on your plan:

MODULE: adder16_beh
- always @(*) ONLY
- sum = $signed({A[15],A}) + $signed({B[15],B});
- ovf = sum[16] ^ sum[15];
- output reg signed [16:0] sum
- output reg ovf

TESTBENCH: tb_adder16

- for loop: 20 pairs, #10, PASS/FAIL
- while loop: 100 iterations, $signed($random), error tracking
- repeat loop: 50 iterations, $signed($random), display
- forever loop: 5ns clk toggle, $finish at 1500ns
- Final error summary

━━

REVIEW PHASE — verify before submitting:

━━

□ adder16_beh has ONLY always @(*) with blocking (=)?
□ sum and ovf declared as output reg?
□ signed on all ports, regs, wires?
□ 17-bit output?
□ Sign extension correct?
□ ovf computed after sum in same always block?
□ All 4 loop types present?
□ $finish present?
□ Final error count printed?

Fix all issues, then output final corrected code only.

...

Design Module:

```
module adder16_beh(
input signed [15:0] A,
input signed [15:0] B,
output reg signed [16:0] sum,
output reg ovf
);

always @(*) begin
sum = $signed({A[15],A}) + $signed({B[15],B});
ovf = sum[16] ^ sum[15];
end

endmodule
```

...

Prompt Strategy: Iterative Correction Prompt

Prompt:

Write a 16-bit signed adder using behavioral modeling through a structured iterative self-correction process.

ABSOLUTE RULE ACROSS ALL ITERATIONS:
The design module must use ONLY always @(*) blocks with blocking assignments (=).
No assign, no structural instantiation, no gate primitives — in any iteration.
All outputs driven from always blocks must be declared as output reg.

ITERATION 1 — Skeleton draft:

Write adder16_beh with ports only:
  input signed [15:0] A, B
  output reg signed [16:0] sum
Use a single always @(*) with: sum = A + B;
(This is intentionally incomplete — you will fix it next.)

ITERATION 2 — Fix sign extension:

Review Iteration 1 for these issues and fix ALL:
  □ Is sum = A + B used? Replace with:
    sum = $signed({A[15],A}) + $signed({B[15],B});
  □ Is sum declared as output reg? Fix if not.
  □ Is always @(*) used? Fix if not.
  □ Is blocking assignment (=) used? Fix if not.
  □ Any assign or structural code present? Remove immediately.
Write corrected version.

ITERATION 3 — Add overflow detection:

Add output reg ovf to module.
Inside existing always @(*) block, after sum computation, add:
  ovf = sum[16] ^ sum[15];
Review:
  □ Is ovf declared as output reg? Fix if not.
  □ Is ovf computed AFTER sum in the always block? Fix order if not.
  □ Manual trace: for A=32767, B=1 → sum=32768 → binary: 01000000000000000
    sum[16]=0, sum[15]=1 → XOR=1 → ovf=1 ✓
Write corrected version.

ITERATION 4 — Basic testbench with for loop:

Write tb_adder16 instantiating adder16_beh. Add only a for loop with 20 corner cases.
Review:
  □ reg signed [15:0] A, B declared?

□ wire signed [16:0] sum declared?
□ wire ovf declared?
□ reg signed [16:0] expected declared?
□ #10 delay before check?
□ expected = $signed(A) + $signed(B) computed?
□ PASS/FAIL compared correctly?
Fix and write corrected version.

ITERATION 5 — Add while and repeat loops:

Add to existing testbench:
WHILE loop: 100 iterations, $signed($random) for A and B
error_count tracking, PASS/FAIL display
REPEAT loop: 50 iterations, $signed($random) inputs
display A, B, sum, ovf each iteration
Review:
□ $signed() cast on all $random uses?
□ #10 delay present in both loops?
□ $display present in both loops?
Fix and write corrected version.

ITERATION 6 — Add forever loop and finalize:

Add:
```
reg clk = 0;
initial forever #5 clk = ~clk;
initial #1000 $finish;
```
Add final summary before $finish:
```
$display("Total errors: %0d", error_count);
```

FINAL REVIEW CHECKLIST:
□ adder16_beh has ONLY always @(*) with blocking (=)?
□ sum and ovf are output reg?
□ signed on ALL ports, regs, wires?
□ 17-bit output?
□ Sign extension correct?
□ ovf after sum in always block?
□ All 4 loops present (for/while/repeat/forever)?
□ $finish present?
□ Error summary printed?
Fix ALL remaining issues.

FINAL OUTPUT: Present ONLY the fully corrected complete Verilog from Iteration 6.

...

Design Module:

```
module adder16_beh(
input signed [15:0] A,
input signed [15:0] B,
output reg signed [16:0] sum,
output reg ovf
);

always @(*) begin
sum = $signed({A[15], A}) + $signed({B[15], B});
ovf = sum[16] ^ sum[15];
end

endmodule
```

...

Prompt Strategy:Hybrid Prompt (All Strategies Combined)

Prompt:

[ROLE]
You are a senior RTL design engineer and Verilog instructor producing a golden behavioral reference implementation for an FPGA design course. Your lab enforces a strict standard: ALL combinational design logic must be written inside always @(*) blocks with blocking assignments (=) ONLY. No assign statements, no structural instantiation, no gate primitives are permitted in any design module.
All outputs driven from always blocks must be declared as output reg.

——

[CHAIN OF THOUGHT — reason through this before coding]

——

1. Behavioral modeling = WHAT the circuit does, expressed in always blocks.
   always @(*) = complete combinational sensitivity list.
   Triggers instantly on any change to A or B. Correct for adder logic.

2. Blocking (=) vs Non-blocking (<=):
   = executes in sequence immediately — correct for combinational always @(*)
   <= schedules at end of time step — only for clocked always @(posedge clk)
   ALWAYS use = inside always @(*).

3. Why output reg?
Verilog rule: any signal assigned inside a procedural block must be reg.
sum and ovf are driven from always @(*) → both must be output reg.

4. Sign extension:
{A[15],A} = 17-bit value with A[15] prepended as the new MSB (sign extension).
$signed() tells Verilog to treat this concatenation as a signed value.
Result: correct 17-bit signed addition preserving two's complement.

5. Overflow detection:
sum[16] = extended carry/sign bit from 17-bit addition.
sum[15] = actual MSB of 16-bit result.
ovf = sum[16]^sum[15]: fires when these disagree = signed overflow occurred.
Verify: 32767+1 → sum=17'b0_1000_0000_0000_0000 → sum[16]=0,sum[15]=1 → ovf=1 ✓
-32768+(-1) → sum=17'b1_0111_1111_1111_1111 → sum[16]=1,sum[15]=0 → ovf=1 ✓
100+200 → sum=300 → sum[16]=0,sum[15]=0 → ovf=0 ✓

[STRICT BEHAVIORAL CONSTRAINT — non-negotiable]

---

adder16_beh must contain EXCLUSIVELY always @(*) with blocking assignments (=).
No assign. No structural instantiation. No gate primitives. No always @(posedge clk).
No non-blocking assignments (<=) anywhere in the design module.

[DO NOT]

- Do not use assign inside adder16_beh
- Do not use always @(posedge clk) in design module
- Do not use non-blocking (<=) inside always @(*)
- Do not use structural instantiation or gate primitives
- Do not declare sum or ovf as wire — must be output reg
- Do not use 16-bit output — must be output reg signed [16:0]
- Do not skip sign extension — must use $signed({A[15],A})+$signed({B[15],B})
- Do not omit ovf — must compute ovf=sum[16]^sum[15] after sum
- Do not use only one loop type — all four required
- Do not use $random without $signed() cast
- Do not skip PASS/FAIL checking in testbench
- Do not leave forever loop without $finish termination
- Do not forget final error summary $display

[FORMAT — deliver in this exact order]

1. adder16_beh module (behavioral only)
2. tb_adder16 testbench

[MODULE SPECIFICATION]

```
module adder16_beh(
  input  signed [15:0] A,       // 16-bit signed input, range -32768 to +32767
  input  signed [15:0] B,       // 16-bit signed input, range -32768 to +32767
  output reg signed [16:0] sum, // 17-bit signed result — captures overflow
  output reg ovf                // 1 = signed overflow occurred
);
  // Combinational behavioral adder
  // always @(*): complete sensitivity list, re-evaluates on any A or B change
  // Blocking (=): sequential execution, correct for combinational logic
  always @(*) begin
    sum = $signed({A[15],A}) + $signed({B[15],B}); // sign-extended addition
    ovf = sum[16] ^ sum[15];                       // overflow detection
  end
endmodule
```

[TESTBENCH SPECIFICATION]

Declarations:

```
  reg  signed [15:0] A, B;
  wire signed [16:0] sum;
  wire ovf;
  integer i, test_count, error_count;
  reg signed [16:0] expected;
```

FOR LOOP — 20 corner case pairs:
(0,0),(1,0),(0,1),(-1,0),(0,-1),(1,1),(-1,-1),(1,-1),
(32767,0),(-32768,0),(32767,1),(-32768,-1),
(32767,32767),(-32768,-32768),(32767,-32768),(-32768,32767),
(16384,16384),(-16384,-16384),(32766,1),(-32767,-1)
Per iteration:

```
  A=val; B=val; #10;
```

```
  expected = $signed(A) + $signed(B);
  if($signed(sum)==expected)
    $display("PASS [%0d] A=%0d B=%0d exp=%0d got=%0d ovf=%b",
            i,$signed(A),$signed(B),expected,$signed(sum),ovf);
  else begin
    error_count=error_count+1;
    $display("FAIL [%0d] A=%0d B=%0d exp=%0d got=%0d ovf=%b",
            i,$signed(A),$signed(B),expected,$signed(sum),ovf);
  end

WHILE LOOP:
  test_count=0; error_count=0;
  while(test_count < 150) begin
    A=$signed($random); B=$signed($random); #10;
    expected=$signed(A)+$signed(B);
    if($signed(sum)!==expected) begin
      error_count=error_count+1;
      $display("FAIL [%0d] A=%0d B=%0d exp=%0d got=%0d ovf=%b",
              test_count,$signed(A),$signed(B),expected,$signed(sum),ovf);
    end else
      $display("PASS [%0d] A=%0d B=%0d sum=%0d ovf=%b",
              test_count,$signed(A),$signed(B),$signed(sum),ovf);
    test_count=test_count+1;
  end

REPEAT LOOP:
  repeat(75) begin
    A=$signed($random); B=$signed($random); #10;
    $display("REPEAT A=%0d B=%0d sum=%0d ovf=%b",
            $signed(A),$signed(B),$signed(sum),ovf);
  end

FOREVER LOOP:
  reg clk=0;
  initial forever #5 clk=~clk;
  initial #2000 $finish;

FINAL SUMMARY:
  $display("═══════════════════════════════════════════════");
  $display("Simulation Complete.");
  $display("Total Errors: %0d", error_count);
  $display("═══════════════════════════════════════════════");
```

[SELF-REVIEW CHECKLIST — fix before outputting]

□ adder16_beh uses ONLY always @(*) with blocking (=)?
□ sum declared as output reg signed [16:0]?
□ ovf declared as output reg?
□ signed keyword on ALL ports, regs, wires?
□ Sign extension: $signed({A[15],A})+$signed({B[15],B})?
□ ovf = sum[16]^sum[15] computed AFTER sum?
□ No assign, no structural, no gate primitives in design?
□ All 4 loop types present in testbench?
□ $signed($random) used everywhere $random appears?
□ $finish present with forever loop terminated?
□ Final error summary $display present?

Fix ALL issues found, then output the final corrected complete code only.

...

Design Module:

```
module adder16_beh(
input signed [15:0] A,
input signed [15:0] B,
output reg signed [16:0] sum,
output reg ovf
);

always @(*) begin
sum = $signed({A[15],A}) + $signed({B[15],B});
ovf = sum[16] ^ sum[15];
end

endmodule
```

...
